P NEEHARIKA-2520090233

PRACTICAL— SESSION 12

Task 1 :- Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <time.h>

#define ITEMS 100000

int *buffer;
int buffer_size;
int in = 0, out = 0;
int produced = 0, consumed = 0;

sem_t empty_slots, full_slots;
pthread_mutex_t mutex;

void *producer(void *arg) {
    for (int i = 0; i < ITEMS; i++) {
        sem_wait(&empty_slots);
        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        in = (in + 1) % buffer_size;
        produced++;

        pthread_mutex_unlock(&mutex);
        sem_post(&full_slots);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < ITEMS; i++) {
        sem_wait(&full_slots);
        pthread_mutex_lock(&mutex);
```

```c
        int item = buffer[out];
        (void)item;
        out = (out + 1) % buffer_size;
        consumed++;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty_slots);
    }
    return NULL;
}

int main(int argc, char *argv[]) {
    pthread_t p, c;
    struct timespec start, end;

    buffer_size = 5;

    if (argc > 1) {
        buffer_size = atoi(argv[1]);
    }

    if (buffer_size < 1 || buffer_size > 100000) {
        printf("Buffer size must be between 1 and 100000.\n");
        return 1;
    }

    buffer = malloc(buffer_size * sizeof(int));
    if (buffer == NULL) {
        perror("malloc");
        return 1;
    }

    sem_init(&empty_slots, 0, buffer_size);
    sem_init(&full_slots, 0, 0);
    pthread_mutex_init(&mutex, NULL);
```

```c
    printf("Producer-Consumer Demonstration\n");
    printf("Buffer size: %d\n", buffer_size);
    printf("Items to produce: %d\n", ITEMS);

    clock_gettime(CLOCK_MONOTONIC, &start);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    clock_gettime(CLOCK_MONOTONIC, &end);

    double elapsed =
        (end.tv_sec - start.tv_sec) +
        (end.tv_nsec - start.tv_nsec) / 1e9;

    printf("Items produced: %d\n", produced);
    printf("Items consumed: %d\n", consumed);
    printf("Elapsed time: %.6f seconds\n", elapsed);
    printf("Throughput: %.0f items/second\n",
           consumed / elapsed);
    printf("Synchronization completed successfully.\n");

    sem_destroy(&empty_slots);
    sem_destroy(&full_slots);
    pthread_mutex_destroy(&mutex);
    free(buffer);

    return 0;
}
```

```
root@HP:~/practical_session12# gcc -pthread task1_producer_consumer.c -o tas
k1_producer_consumer
root@HP:~/practical_session12# ./task1_producer_consumer 5
Producer-Consumer Demonstration
Buffer size: 5
Items to produce: 100000
Items produced: 100000
Items consumed: 100000
Elapsed time: 2.415928 seconds
Throughput: 41392 items/second
Synchronization completed successfully.
root@HP:~/practical_session12# ./task1_producer_consumer 20
Producer-Consumer Demonstration
Buffer size: 20
Items to produce: 100000
Items produced: 100000
Items consumed: 100000
Elapsed time: 0.582976 seconds
Throughput: 171534 items/second
Synchronization completed successfully.
root@HP:~/practical_session12#
```

Task 2 :- Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource_A;
pthread_mutex_t resource_B;

void *thread1(void *arg) {
    printf("Thread 1: Requesting Resource A\n");

    pthread_mutex_lock(&resource_A);
    printf("Thread 1: Acquired Resource A\n");

    usleep(100000);

    printf("Thread 1: Requesting Resource B\n");
    pthread_mutex_lock(&resource_B);

    printf("Thread 1: Acquired Resource B\n");
    printf("Thread 1: Completed work\n");

    pthread_mutex_unlock(&resource_B);
    pthread_mutex_unlock(&resource_A);

    return NULL;
}

void *thread2(void *arg) {
    printf("Thread 2: Requesting Resource B\n");
```

```
 GNU nano 7.2                    task2_deadlock.c *

    /* Resource ordering: acquire A before B */
    pthread_mutex_lock(&resource_A);
    printf("Thread 2: Acquired Resource A\n");

    usleep(100000);

    printf("Thread 2: Requesting Resource B\n");
    pthread_mutex_lock(&resource_B);

    printf("Thread 2: Acquired Resource B\n");
    printf("Thread 2: Completed work\n");

    pthread_mutex_unlock(&resource_B);
    pthread_mutex_unlock(&resource_A);

    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&resource_A, NULL);
    pthread_mutex_init(&resource_B, NULL);

    printf("Deadlock Prevention Demonstration\n");
    printf("Both threads acquire resources in A-B order.\n\n");

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("\nBoth threads completed without deadlock.\n");

    printf("\nFour necessary deadlock conditions:\n");
```

```
    printf("\nFour necessary deadlock conditions:\n");
    printf("1. Mutual exclusion\n");
    printf("2. Hold and wait\n");
    printf("3. No preemption\n");
    printf("4. Circular wait\n");

    printf("\nCircular wait is prevented by resource ordering.\n");

    pthread_mutex_destroy(&resource_A);
    pthread_mutex_destroy(&resource_B);

    return 0;
}
```

```
root@HP:~/practical_session12# gcc -pthread task2_deadlock.c -o task2_deadlo
ck
root@HP:~/practical_session12# ./task2_deadlock
Deadlock Prevention Demonstration
Both threads acquire resources in A-B order.

Thread 1: Requesting Resource A
Thread 1: Acquired Resource A
Thread 2: Requesting Resource B
Thread 1: Requesting Resource B
Thread 1: Acquired Resource B
Thread 1: Completed work
Thread 2: Acquired Resource A
Thread 2: Requesting Resource B
Thread 2: Acquired Resource B
Thread 2: Completed work

Both threads completed without deadlock.

Four necessary deadlock conditions:
1. Mutual exclusion
2. Hold and wait
3. No preemption
4. Circular wait

Circular wait is prevented by resource ordering.
root@HP:~/practical_session12#
```

Observation :- The Producer-Consumer problem was implemented using semaphores and threads, and resource ordering was demonstrated as a method of preventing deadlocks.